Exploratory Data Analysis: Covariation

PSY 410: Data Science for Psychology

Dr. Sara Weston

2026-04-29

Setup

From variation to covariation

Psychology is about relationships

Last time, you explored how individual variables behave — distributions, outliers, missing data.

. . .

But psychology is about *relationships*:

- Does **treatment** predict **depression**?
- Does **age** relate to **reaction time**?
- Does **anxiety** co-occur with **insomnia**?

. . .

Today we learn to **see** those relationships in data — before testing them statistically.

Connect back to last session: "Last time we looked at one variable at a time. But the questions you actually care about in psychology involve relationships." Spend ~2 minutes. The three example questions (treatment/depression, age/RT, anxiety/insomnia) are deliberately chosen to span different variable type pairings — point out that we'll learn a different plot type for each.

Categorical + Continuous

Example: Mental health by treatment group

```
# Simulated therapy outcome data
therapy_data <- tibble(
  condition = rep(c("Control", "CBT", "Mindfulness"), each = 50),
  depression_post = c(
    rnorm(50, mean = 18, sd = 5),  # Control
    rnorm(50, mean = 12, sd = 5),  # CBT
    rnorm(50, mean = 14, sd = 5)   # Mindfulness
  )
)
```

Example: Mental health by treatment group

```
glimpse(therapy_data)
```

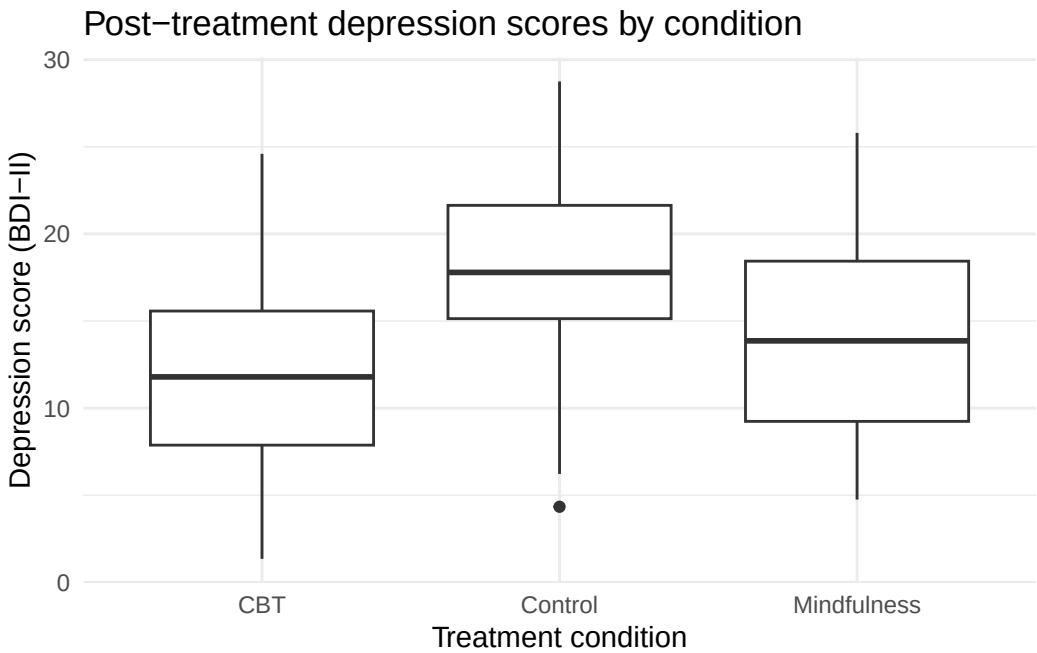
```
Rows: 150
Columns: 2
$ condition       <chr> "Control", "Control", "Control", "Control", "Control",~
$ depression_post <dbl> 21.907250, 12.788407, 26.363560, 17.562985, 24.315121,~
```

Walk through the simulated data structure. Emphasize that this is simulated — we chose the means and SDs on purpose so the CBT group looks better. In real research, you wouldn't know the answer in advance. The dataset is small (150 rows) and simple so students can focus on the visualization, not the data wrangling. Students may ask about BDI-II — it's the Beck Depression Inventory, a widely used clinical measure.

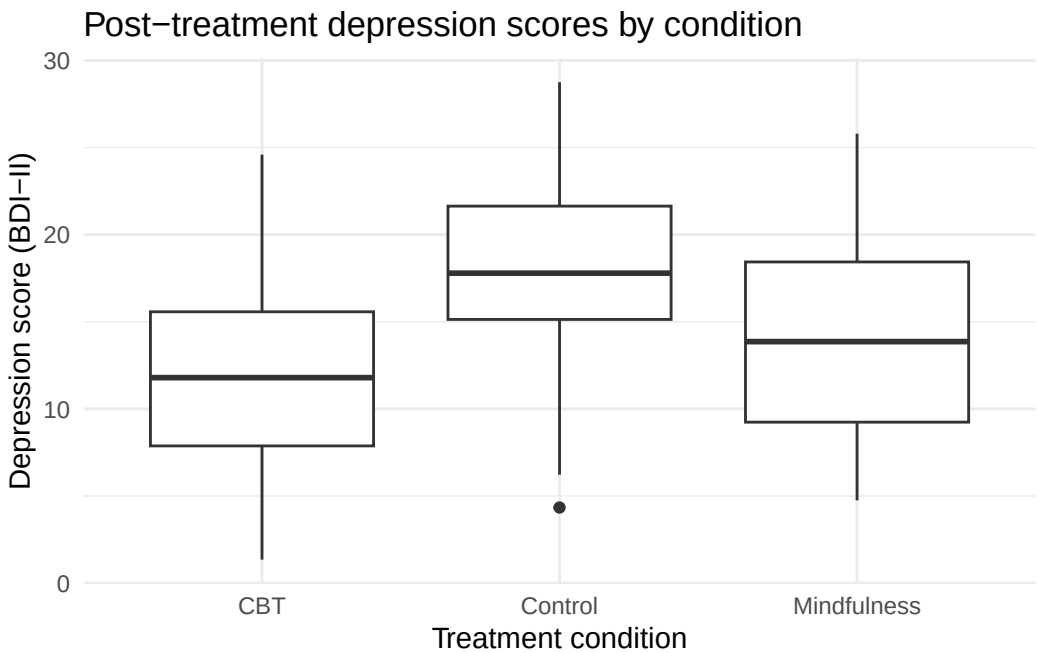
Boxplots: The classic choice

```
ggplot(therapy_data, aes(x = condition, y = depression_post)) +
  geom_boxplot() +
  labs(
    title = "Post-treatment depression scores by condition",
    x = "Treatment condition",
```

```
    y = "Depression score (BDI-II)"
  ) +
  theme_minimal()
```



What boxplots show



- **Line in middle:** median
- **Box:** 25th to 75th percentile (IQR)
- **Whiskers:** extend to 1.5 × IQR
- **Dots:** outliers beyond whiskers

. . .

Great for comparing distributions, but they hide the actual data points.

Spend ~2 minutes on the anatomy of boxplots. Students may have seen boxplots in stats but not know the mechanics. Draw attention to the specific components: "The line in the middle is NOT the mean — it's the median." This is a common misconception. Ask students: "Based on this boxplot, which treatment looks most effective?" (CBT has the lowest median.)

The problem with boxplots

Boxplots summarize, but they hide important information:

- **The actual distribution shape** (is it bimodal? skewed?)
- **Individual data points** (how many observations are there?)
- **The raw data** (where do specific values fall?)

. . .

We can do better.

This slide motivates the next section on raincloud plots. The key point: boxplots hide bimodality, sample size, and the actual distribution shape. You can show the classic "same boxplot, different distributions" example verbally — two very different distributions can produce identical boxplots. If students push back ("but boxplots are fine for homework"), acknowledge that boxplots are quick and useful, but for publications and presentations, we want more information.

Raincloud plots

The modern psych visualization

Raincloud plots combine three elements:

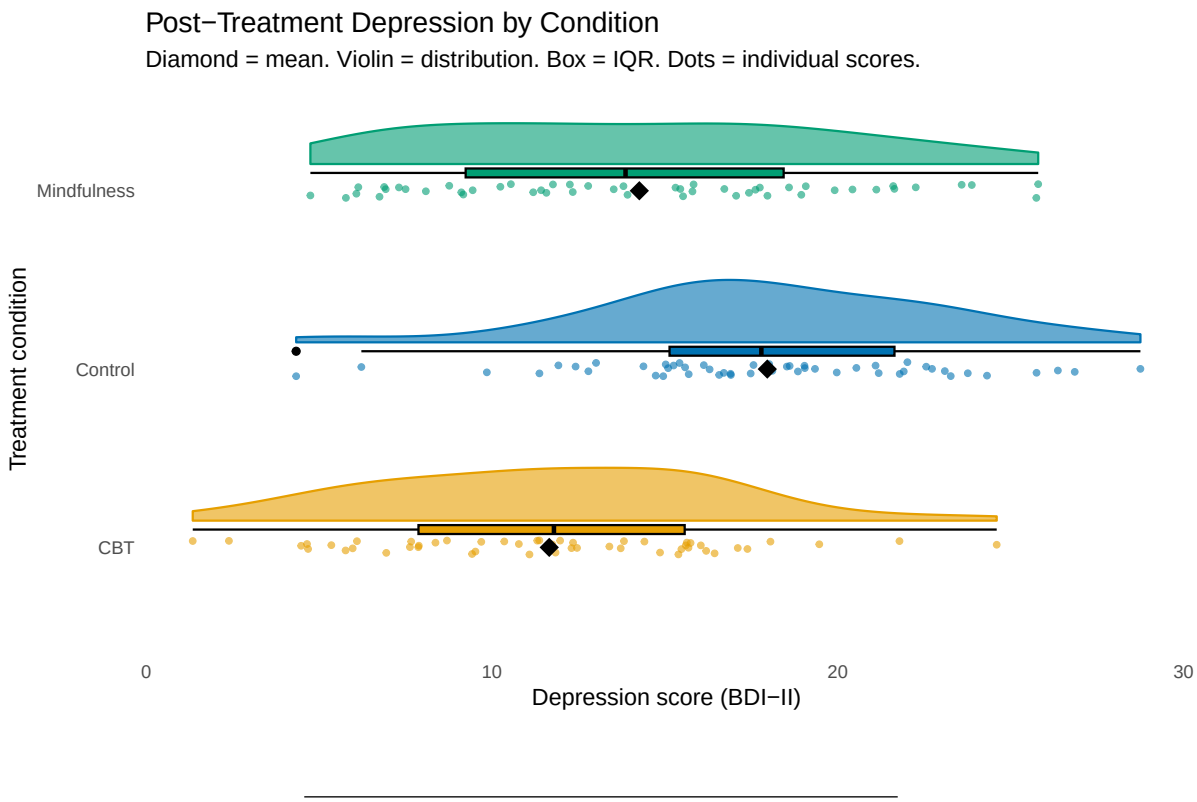
1. **Violin** (distribution shape)
2. **Boxplot** (summary stats)
3. **Jittered points** (individual data)

They're increasingly popular in psychology publications because they show **everything**.

Use the `ggrain` package. (More details here.)

Spend ~3 minutes on this conceptual intro. Raincloud plots are increasingly expected in psychology journals — if students go on to do research, they'll encounter these. The name comes from the shape: the violin is the "cloud" and the jittered points are the "rain." The ggrain package makes them easy; without it, you'd need to layer violin + boxplot + jitter manually. Students do NOT need to memorize the ggrain syntax for the assignment — boxplots are fine — but exposure here is valuable.

Building a raincloud



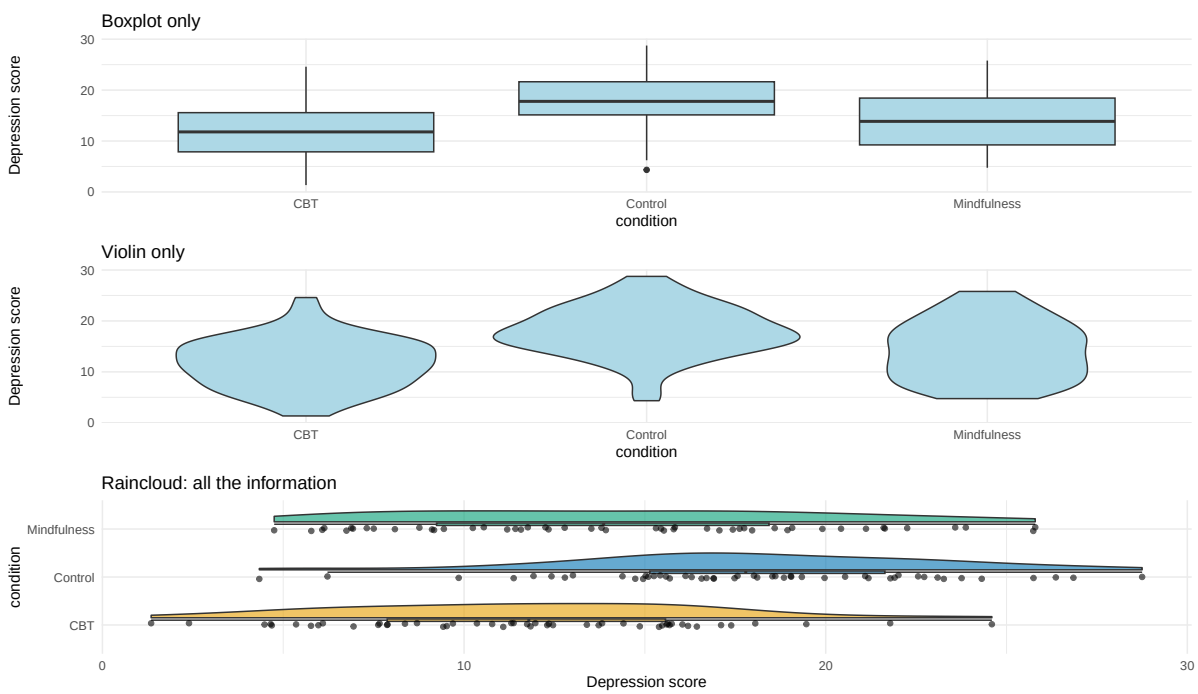

```
library(ggrain)
therapy_data |>
  ggplot(aes(
    x = condition,
    y = depression_post,
    fill = condition,
    color = condition)) +
  # geom_rain creates all parts of your raincloud
  geom_rain(
    alpha = .6,
    # change just the boxplot part
    boxplot.args = list(color = "black")) +
  # The mean
  stat_summary(fun = mean, geom = "point", shape = 18, size = 5, color = "black") +
  scale_fill_manual(
    values = c("Control" = "#0072B2", "CBT" = "#E69F00", "Mindfulness" = "#009E73")) +
  scale_color_manual(
```

```
    values = c("Control" = "#0072B2", "CBT" = "#E69F00", "Mindfulness" = "#009E73")) +
  labs(
    title = "Post-Treatment Depression by Condition",
    subtitle = "Diamond = mean. Violin = distribution. Box = IQR. Dots = individual scores."
    x = "Treatment condition",
    y = "Depression score (BDI-II)"
  ) +
  coord_flip() +
  theme_minimal(base_size = 14) +
  theme(
    legend.position = "none",
    panel.grid = element_blank())
```

Walk through the code on this slide since the previous slide just showed the output. Point out the key pieces: geom_rain() does the heavy lifting, stat_summary adds the mean diamond, and coord_flip() makes it horizontal. Don't get bogged down in the color scale code. The important takeaway is that geom_rain() gives you violin + box + points in one function call.

Why rainclouds are great

Compare these three views of the **same data**:



When to use which

Plot type	Shows	Best for
Boxplot	Median, IQR, outliers	Quick comparison, large datasets
Raincloud	Distribution + summary + raw data	Psychology papers, presentations, publications
Violin + jitter	Distribution + raw data	Alternative to raincloud, simpler to code

Quick reference slide — ~30 seconds. Emphasize that boxplots are still perfectly fine for EDA and assignments. Rainclouds are the ideal for final presentations and publications. Violin + jitter is a good middle ground if ggrain isn't installed.

Pair coding break

Your turn: Compare by gender

Using the therapy data, explore whether depression scores differ by gender:

1. Add a `gender` variable to the data (use `sample()` to randomly assign "Male", "Female", "Non-binary")
2. Create a visualization showing depression scores by gender
3. Try at least two different geom types
4. Add appropriate labels

Time: 10 minutes

Circulate during this exercise. The main challenge is step 1: students need to add a column using mutate + sample(). Some may struggle with the sample() syntax — remind them of the arguments: values to choose from, number to draw, and replace = TRUE. If pairs finish early, ask them to add a facet_wrap(~condition) to see gender differences within each treatment group. Common error: forgetting replace = TRUE when the number of rows exceeds the number of categories.

Categorical + Categorical

Example: Diagnosis by gender

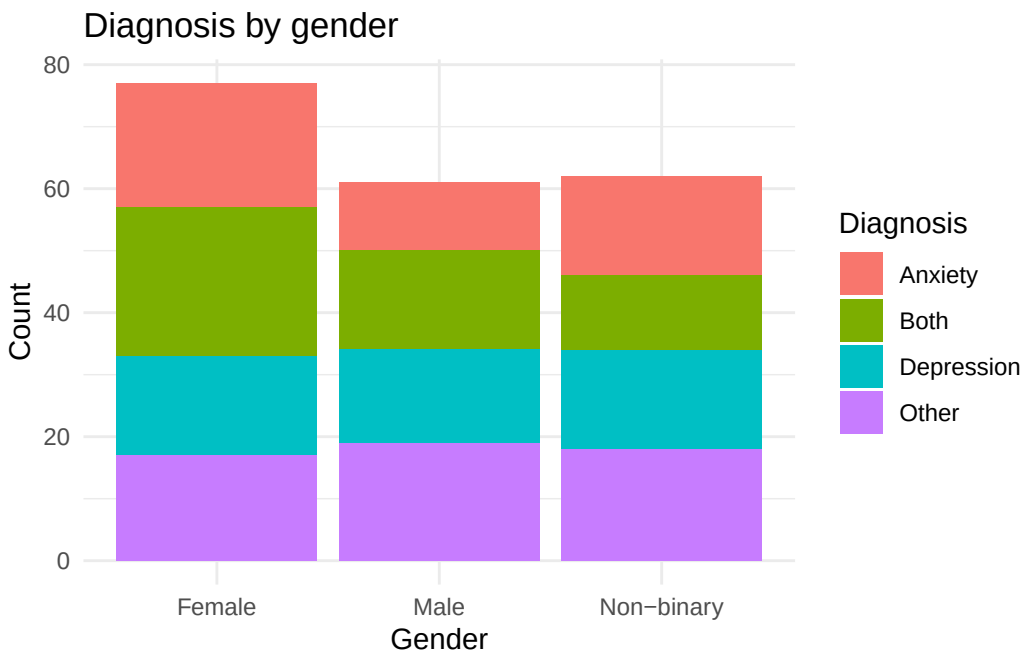
```r
# Simulated diagnostic data
diagnosis_data <- tibble(
  gender = sample(c("Male", "Female", "Non-binary"), 200, replace = TRUE),
  diagnosis = sample(c("Depression", "Anxiety", "Both", "Other"),
                     200, replace = TRUE)
)

head(diagnosis_data)
```

```
# A tibble: 6 x 2
  gender     diagnosis
  <chr>      <chr>
1 Non-binary Anxiety
2 Male       Anxiety
3 Non-binary Other
4 Male       Anxiety
5 Female     Both
6 Female     Both
```

Option 1: Stacked bar chart

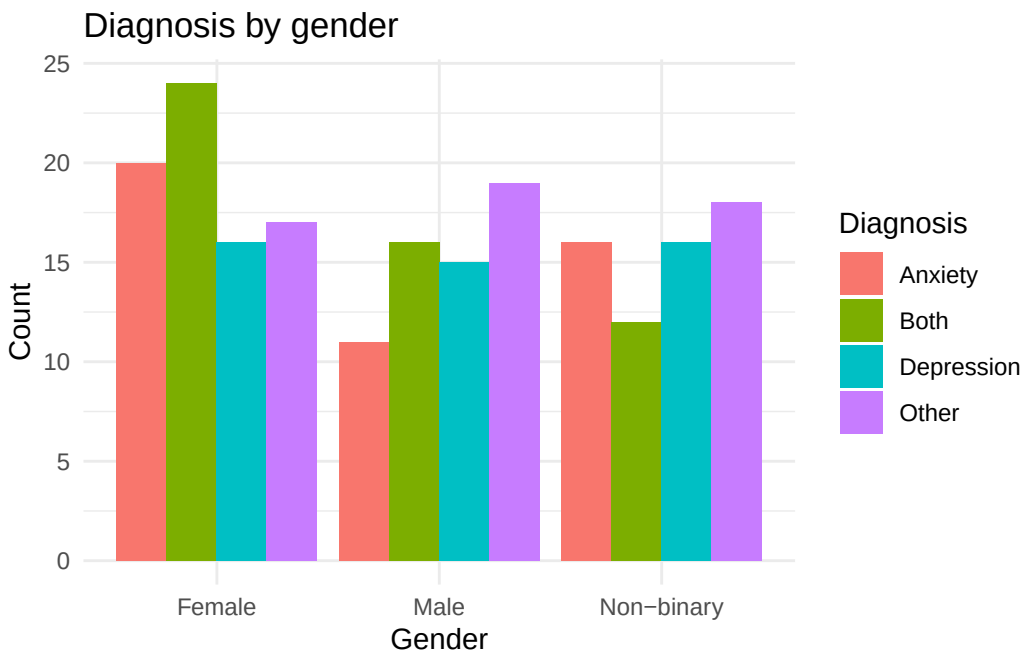
The next five slides show five options for the same data. Move through them at a moderate pace (~1 minute each). For stacked bars, point out the problem: it's hard to compare the sizes of non-bottom segments across groups because they don't share a common baseline. This is a perceptual limitation, not a coding one.

```r
ggplot(diagnosis_data, aes(x = gender, fill = diagnosis)) +
  geom_bar() +
  labs(
    title = "Diagnosis by gender",
    x = "Gender",
    y = "Count",
    fill = "Diagnosis"
  ) +
  theme_minimal()
```



Option 2: Side-by-side bars

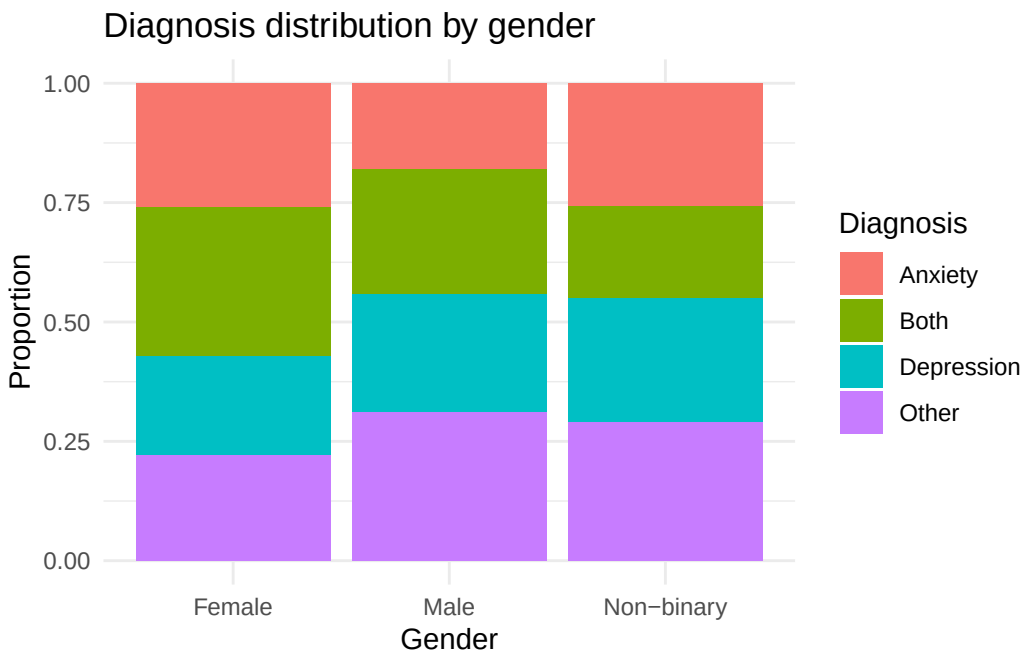
```
ggplot(diagnosis_data, aes(x = gender, fill = diagnosis)) +
  geom_bar(position = "dodge") +
  labs(
    title = "Diagnosis by gender",
    x = "Gender",
    y = "Count",
    fill = "Diagnosis"
  ) +
  theme_minimal()
```



Option 3: Proportions

Highlight the difference between position = "dodge" (previous slide) and position = "fill" (this slide). "fill" normalizes each bar to 100%, which makes it easy to compare proportions across groups but hides differences in group sizes. Ask: "If one gender group has 10 people and another has 100, this chart makes them look equally important. Is that a problem?" This gets students thinking about when proportions vs. counts are appropriate.

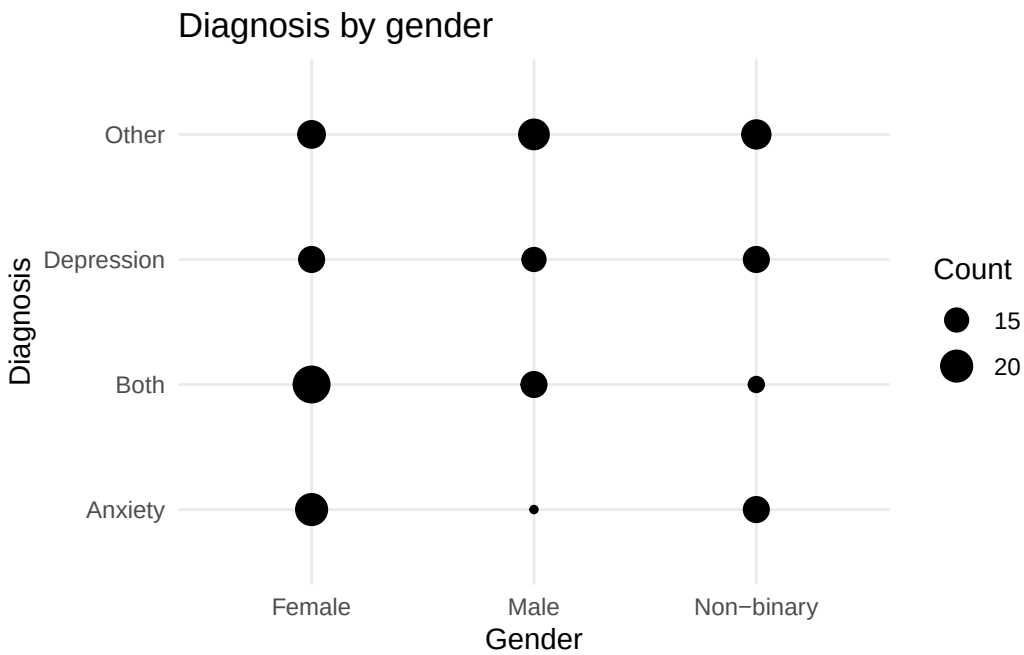
```
ggplot(diagnosis_data, aes(x = gender, fill = diagnosis)) +
  geom_bar(position = "fill") +
  labs(
    title = "Diagnosis distribution by gender",
    x = "Gender",
    y = "Proportion",
    fill = "Diagnosis"
  ) +
  theme_minimal()
```



Option 4: geom_count()

Shows the *size* of overlaps:

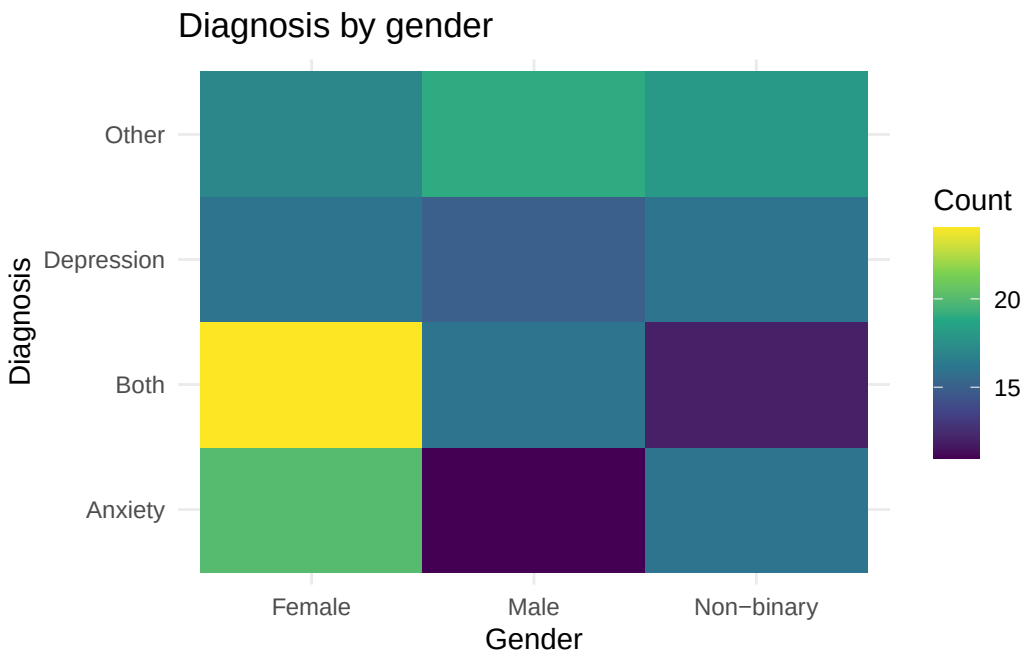
```
ggplot(diagnosis_data, aes(x = gender, y = diagnosis)) +
  geom_count() +
  labs(
    title = "Diagnosis by gender",
    x = "Gender",
    y = "Diagnosis",
    size = "Count"
  ) +
  theme_minimal()
```



Option 5: Tile with counts

geom_tile() is essentially a heatmap. Note that this requires pre-computing the counts with count() before piping into ggplot — this is different from the previous options where geom_bar() computed counts automatically. The viridis color scale is colorblind-friendly. This approach works best with many categories on both axes; with only 3x4, a bar chart is probably clearer. Still worth showing so students have it in their toolkit.

```
diagnosis_counts <- diagnosis_data |>
  count(gender, diagnosis)

ggplot(diagnosis_counts, aes(x = gender, y = diagnosis, fill = n)) +
  geom_tile() +
  scale_fill_viridis_c() +
  labs(
    title = "Diagnosis by gender",
    x = "Gender",
    y = "Diagnosis",
    fill = "Count"
  ) +
  theme_minimal()
```



Continuous + Continuous

The classic: Scatterplots

```
# Simulated reaction time data
rt_data <- tibble(
  age = runif(100, 18, 70),
  reaction_time = 200 + age * 3 + rnorm(100, 0, 40)
)

glimpse(rt_data)
```

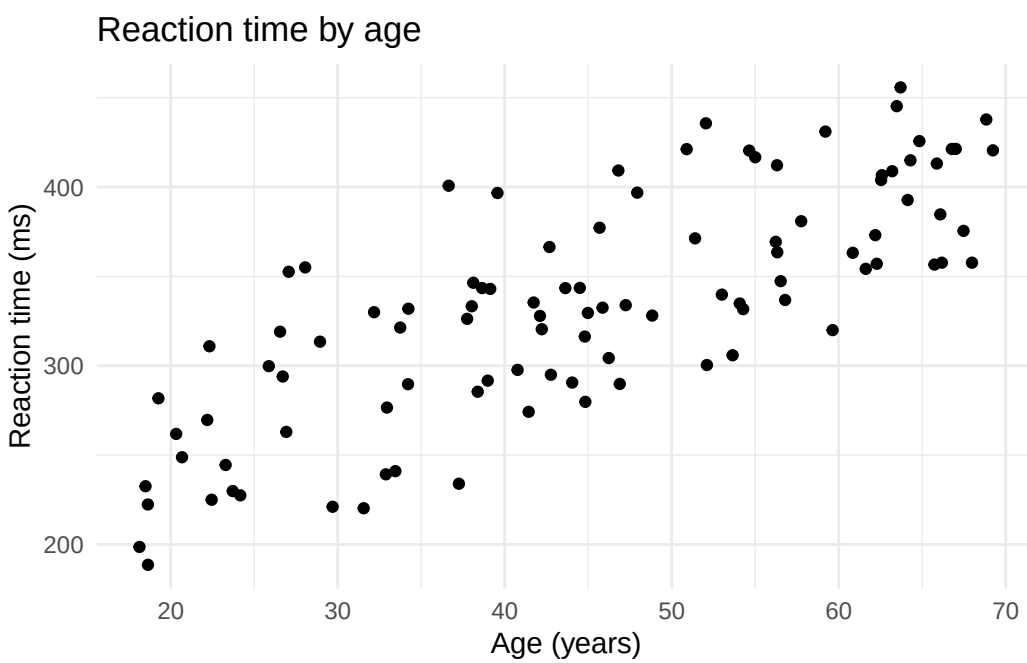
```
Rows: 100
Columns: 2
$ age           <dbl> 41.73548, 27.06823, 59.63581, 34.23378, 37.25926, 29.698~
$ reaction_time <dbl> 335.3821, 352.5746, 319.9258, 331.9031, 233.9706, 221.07~
```

Basic scatterplot

Scatterplots should feel familiar from stats classes, but many students haven't made one in R yet. Walk through the aes() mapping: x goes on the horizontal axis, y goes on the vertical.

Ask: "What would you predict about the relationship between age and reaction time?" Most students will guess that older people are slower, which matches the data. Spend ~2 minutes here.

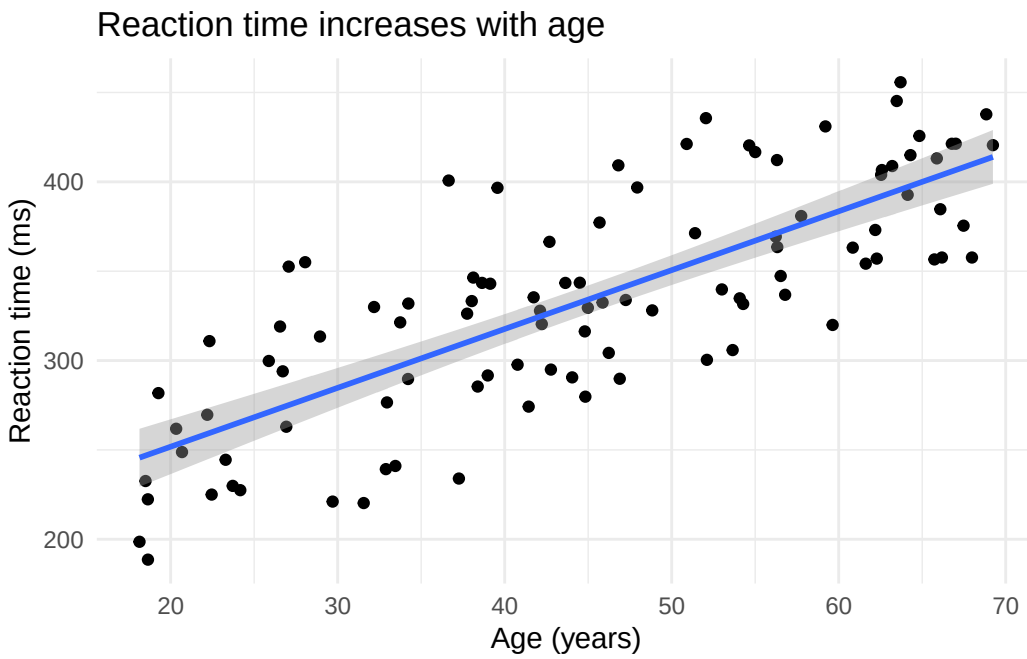
```
ggplot(rt_data, aes(x = age, y = reaction_time)) +
  geom_point() +
  labs(
    title = "Reaction time by age",
    x = "Age (years)",
    y = "Reaction time (ms)"
  ) +
  theme_minimal()
```



Add a trend line

geom_smooth(method = "lm") adds a linear regression line. Students will see lm() again in Session 17 — for now, just explain that it draws the "best-fit" straight line through the data. The gray band is the confidence interval (se = TRUE). Students often ask "what does the gray area mean?" — a simple answer is "where we think the true line probably falls." Don't go deep into regression mechanics here.

```r
ggplot(rt_data, aes(x = age, y = reaction_time)) +
  geom_point() +
  geom_smooth(method = "lm", se = TRUE) +
  labs(
    title = "Reaction time increases with age",
    x = "Age (years)",
    y = "Reaction time (ms)"
  ) +
  theme_minimal()
```



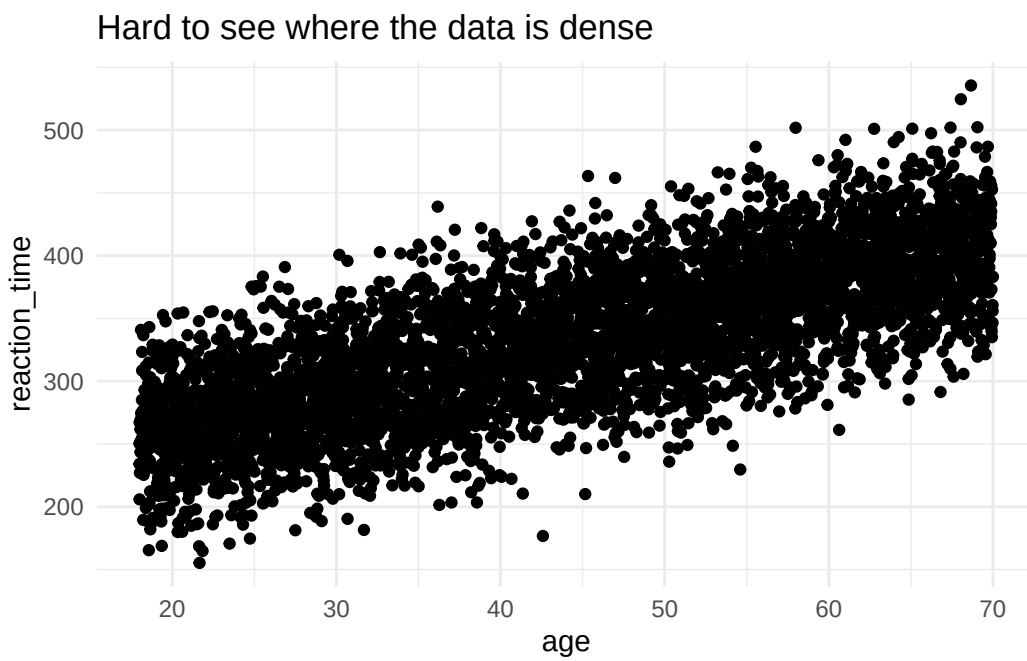
The problem: Overplotting

With lots of data, points overlap and hide the true density:

```r
# Lots of data
big_rt_data <- tibble(
  age = runif(5000, 18, 70),
  reaction_time = 200 + age * 3 + rnorm(5000, 0, 40)
)
```

Overplotting problem demonstrated

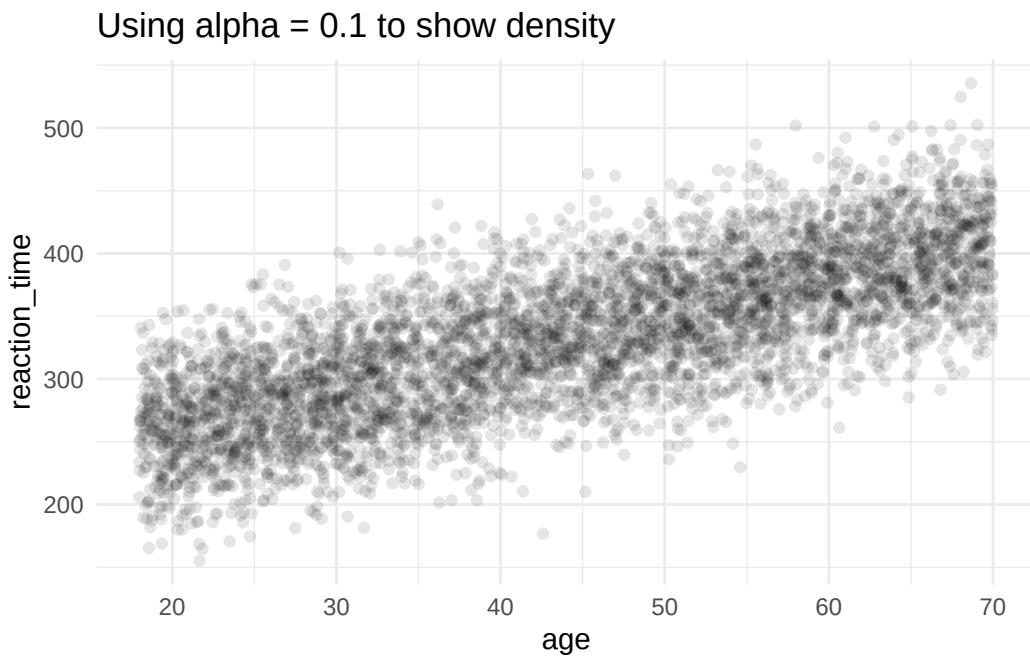
```
ggplot(big_rt_data, aes(x = age, y = reaction_time)) +
  geom_point() +
  labs(title = "Hard to see where the data is dense") +
  theme_minimal()
```



The plot looks like a solid blob. Ask: "Can you tell where most of the data is?" (No — that's the problem.) With 5,000 points, geom_point() alone fails. This motivates the next three solutions. In psychology, overplotting is common with large survey datasets or reaction time studies with many trials.

Solution 1: Transparency (alpha)

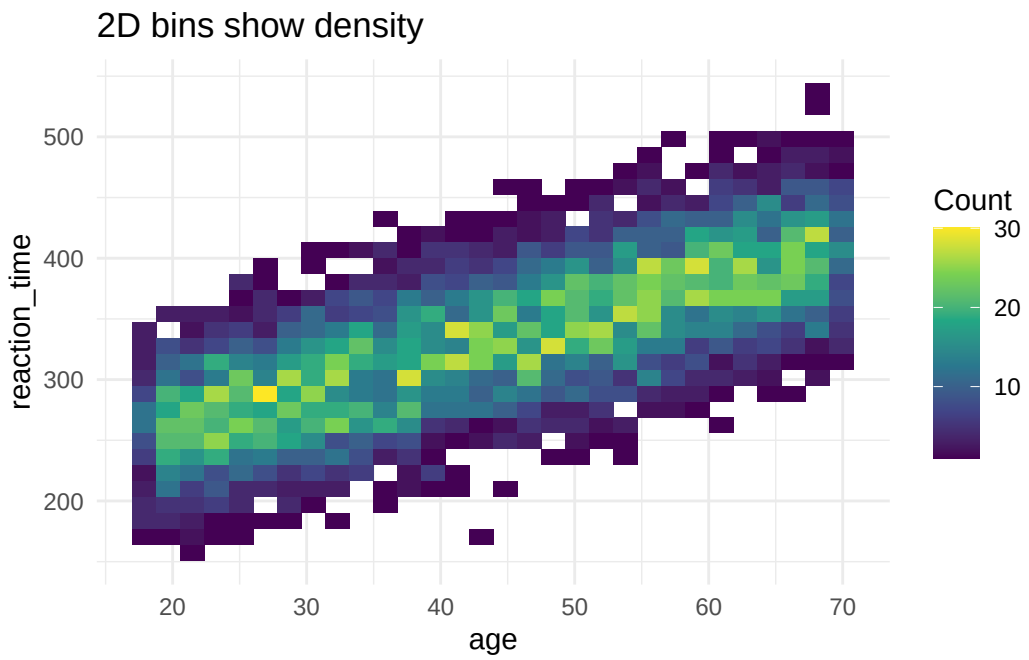
```
ggplot(big_rt_data, aes(x = age, y = reaction_time)) +
  geom_point(alpha = 0.1) +
  labs(title = "Using alpha = 0.1 to show density") +
  theme_minimal()
```



Alpha is the simplest fix and often the best first attempt. Explain: alpha = 0.1 means each point is 90% transparent, so areas with many overlapping points appear darker. Students should experiment with different alpha values — 0.1 is good for 5000 points, but 0.5 might be better for 200 points. This is the overplotting solution they'll use most often.

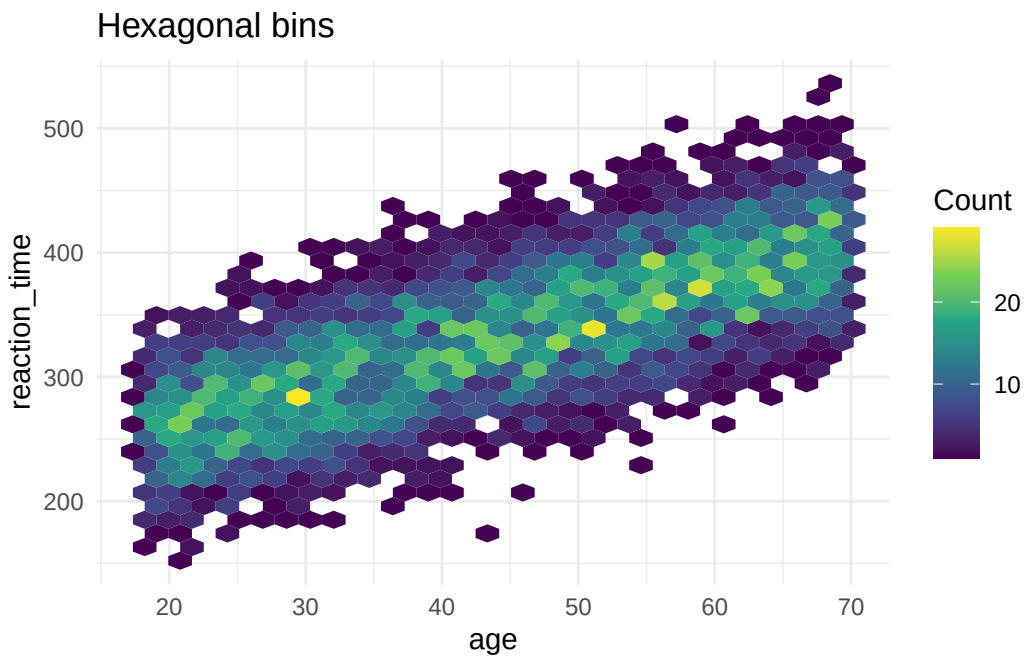
Solution 2: geom_bin2d()

```
ggplot(big_rt_data, aes(x = age, y = reaction_time)) +
  geom_bin2d() +
  scale_fill_viridis_c() +
  labs(
    title = "2D bins show density",
    fill = "Count"
  ) +
  theme_minimal()
```



Solution 3: geom_hex()

```
ggplot(big_rt_data, aes(x = age, y = reaction_time)) +
  geom_hex() +
  scale_fill_viridis_c() +
  labs(
    title = "Hexagonal bins",
    fill = "Count"
  ) +
  theme_minimal()
```



Correlation coefficient

This is a preview of Session 17 (Correlation and Regression). Keep it brief — ~2 minutes. The goal is just to connect the visual pattern they saw in the scatterplot to a single number. Most students have encountered correlation coefficients in stats, so this should feel familiar. Emphasize the dollar-sign syntax (rt_data$age) as an alternative to piping — cor() expects vectors, not a data frame. Students often ask "is this r significant?" — save that for Session 17.

A single number summary of the linear relationship:

```
cor(rt_data$age, rt_data$reaction_time)
```

```
[1] 0.7815725
```

. . .

> **Note**
>
> - **r = 1:** perfect positive relationship
> - **r = 0:** no linear relationship
> - **r = -1:** perfect negative relationship

. . .

But always plot your data first! (See: Anscombe's Quartet)

If you have time, briefly describe Anscombe's Quartet: four datasets with identical means, standard deviations, and correlations that look completely different when plotted. This powerfully reinforces the "always visualize" message. You can also mention the Datasaurus Dozen as a modern, fun version of the same idea.

Patterns and models

What patterns tell us

When you see covariation, ask:

1. **Could it be coincidence?** (Maybe, especially with small samples)
2. **What's the mechanism?** (How are these variables related?)
3. **Is there a confound?** (Could a third variable explain both?)

. . .

> ⚠ Warning
>
> **Correlation Causation**
> Covariation suggests a relationship, but doesn't prove one variable *causes* the other.

Spend ~3 minutes here — this is conceptually the most important slide in the deck. Walk through each question with the age/reaction-time example: (1) Could it be coincidence? With 100 data points and a clear trend, probably not. (2) What's the mechanism? Cognitive processing speed declines with age. (3) Is there a confound? Maybe older participants have different levels of education, health, or motivation. The correlation vs. causation callout connects directly to what they learned in stats — reinforce it here.

Real data example

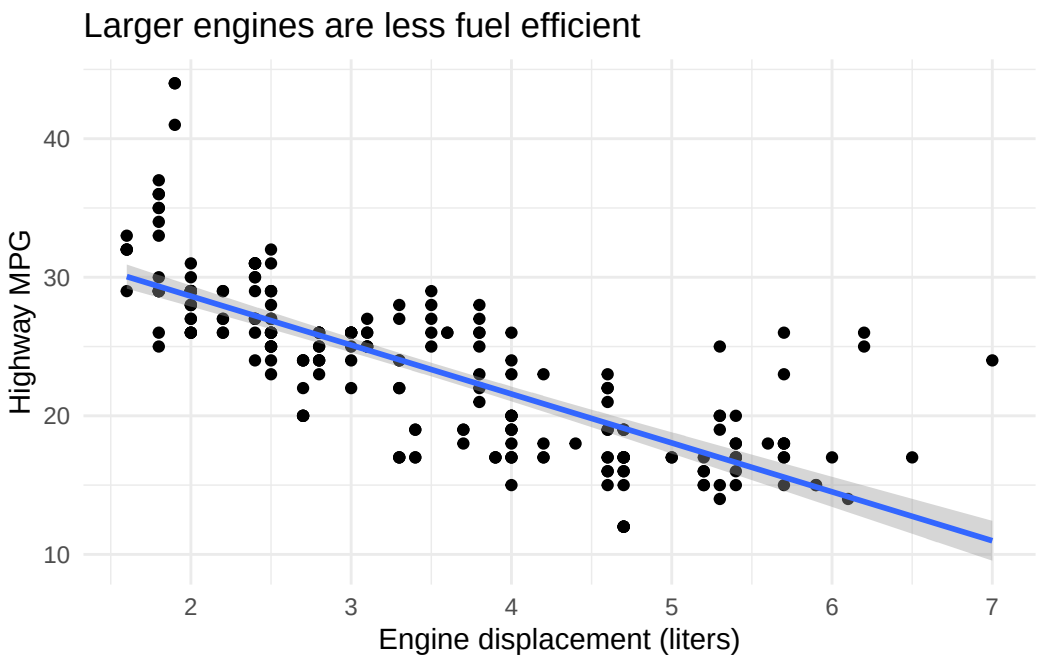
Let's explore a real dataset: `mpg` (fuel economy data)

```
glimpse(mpg)
```

```
Rows: 234
Columns: 11
$ manufacturer <chr> "audi", "audi", "audi", "audi", "audi", "audi", "audi", "~
$ model        <chr> "a4", "a4", "a4", "a4", "a4", "a4", "a4", "a4 quattro", "~
$ displ        <dbl> 1.8, 1.8, 2.0, 2.0, 2.8, 2.8, 3.1, 1.8, 1.8, 2.0, 2.0, 2.~
$ year         <int> 1999, 1999, 2008, 2008, 1999, 1999, 2008, 1999, 1999, 200~
$ cyl          <int> 4, 4, 4, 4, 6, 6, 6, 4, 4, 4, 4, 6, 6, 6, 6, 6, 6, 8, 8, ~
$ trans        <chr> "auto(l5)", "manual(m5)", "manual(m6)", "auto(av)", "auto~
$ drv          <chr> "f", "f", "f", "f", "f", "f", "f", "4", "4", "4", "4", "4~
$ cty          <int> 18, 21, 20, 21, 16, 18, 18, 18, 16, 20, 19, 15, 17, 17, 1~
$ hwy          <int> 29, 29, 31, 30, 26, 26, 27, 26, 25, 28, 27, 25, 25, 25, 2~
$ fl           <chr> "p", "p", "p", "p", "p", "p", "p", "p", "p", "p", "p", "p~
$ class        <chr> "compact", "compact", "compact", "compact", "compact", "c~
```

Question 1: Does engine size affect fuel economy?

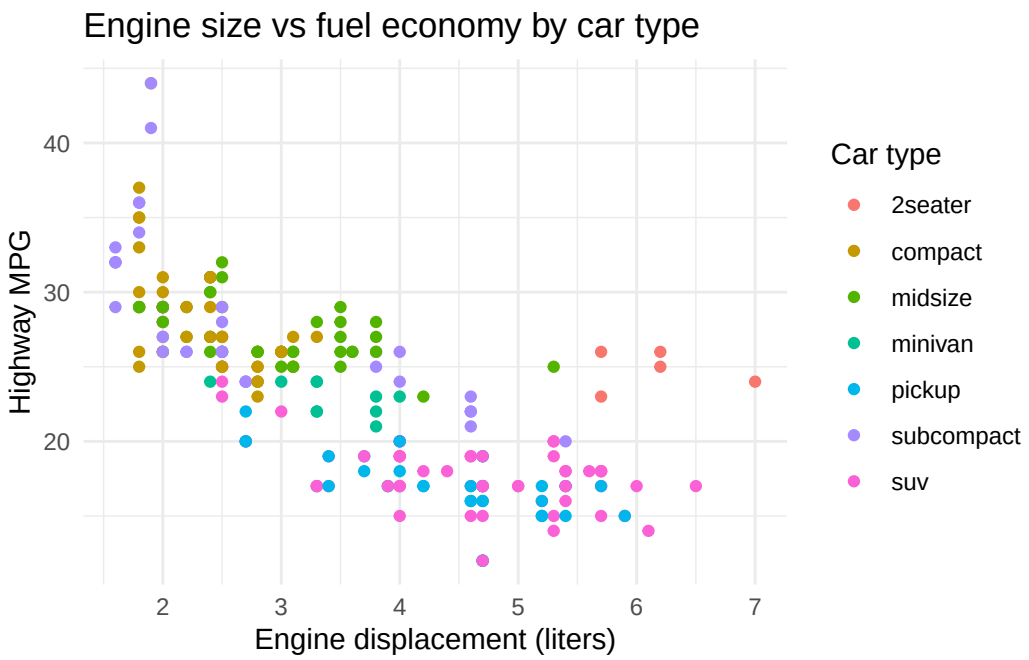
```r
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point() +
  geom_smooth(method = "lm") +
  labs(
    title = "Larger engines are less fuel efficient",
    x = "Engine displacement (liters)",
    y = "Highway MPG"
  ) +
  theme_minimal()
```



Question 2: Does this vary by car type?

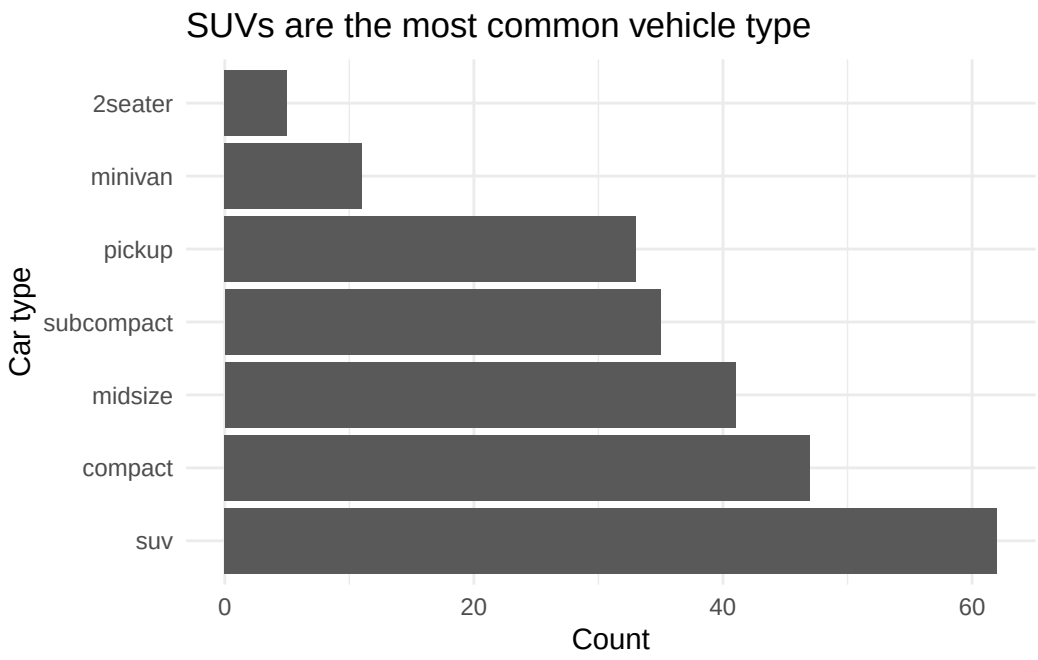
This slide demonstrates a key EDA move: adding a third variable via color. Walk through the insight: some car types (2seater, subcompact) break the overall trend — they have small engines but aren't particularly fuel-efficient, or they have large engines but are surprisingly efficient. This is a confound in action: the relationship between engine size and fuel economy depends on the type of car.

```
ggplot(mpg, aes(x = displ, y = hwy, color = class)) +
  geom_point() +
  labs(
    title = "Engine size vs fuel economy by car type",
    x = "Engine displacement (liters)",
    y = "Highway MPG",
    color = "Car type"
  ) +
  theme_minimal()
```



Question 3: Which car types are most common?

```
ggplot(mpg, aes(y = fct_infreq(class))) +
  geom_bar() +
  labs(
    title = "SUVs are the most common vehicle type",
    x = "Count",
    y = "Car type"
  ) +
  theme_minimal()
```



End-of-deck exercise

Your final project proposal

For your final project, you'll need to:

1. Choose a dataset with at least 2-3 variables of interest
2. Formulate 2-3 research questions about relationships in the data
3. Plan visualizations to explore those relationships

Exercise: Start exploring potential datasets. Find one that interests you and create 2-3 exploratory visualizations showing different types of covariation (categorical + continuous, continuous + continuous, etc.).

This will form the basis of your proposal, **due today!**

Give students ~10 minutes of work time here. This exercise doubles as final project proposal prep — the proposal is due today, so this is their chance to get started or polish what they have. If students don't have a dataset yet, suggest palmerpenguins, mpg, or the bfi data from last session. Circulate and help students think about which variable combinations are interesting. Remind them that the proposal only needs 2-3 research questions and planned visualizations, not finished analyses.

Wrapping up

Key takeaways

1. **Covariation = relationships** between variables
2. **Different plot types** for different variable combinations:
   - Categorical + continuous: boxplot, raincloud plots
   - Categorical + categorical: bar charts, tiles, geom_count()
   - Continuous + continuous: scatterplot, 2D bins, hex
3. **Raincloud plots** are the gold standard for psychology — they show distribution, summary stats, and raw data
4. **Watch for overplotting** — use alpha, jitter, or binning
5. **Always visualize first** before computing correlations
6. **Patterns suggest but don't prove** causation

Quick summary — ~1 minute. If you're short on time, let students read this slide themselves. The most important takeaway is point 3 (raincloud plots) and point 6 (correlation is not causation). Remind them that Assignment 5 will ask them to use several of these plot types on a real dataset.

Before next class

Read:

- R4DS Ch 12: Logical vectors
- R4DS Ch 13: Numbers

Do:

- Submit Assignment 5
- Submit your Final Project Proposal (due today!)
- Start thinking about how you'll compute scale scores (next session)

The one thing to remember

Relationships hide in data. Your job is to make them visible — carefully, honestly.

See you Monday for data types and scale scoring!

End on this closing line. Remind students: Assignment 4 is due, Assignment 5 (EDA) is now assigned, and the Final Project Proposal is due today. Next session shifts from visualization to data types — logical vectors, numbers, and computing scale scores (like averaging personality

items into composite scores). That will connect directly back to the BFI data they've been exploring.